

2025 – 2026

esprit
Se former autrement

Rapport de Stage d'Ingénieur

Sujet: Conception et développement d'un système de test autonome pour agents conversationnels basé sur OpenAI Swarm et Ollama

Élaboré par : Amine Manai

Encadré par : Ahmed Rebai

Effectué au sein de:



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

Remerciements

Au terme de ce stage d'ingénieur, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de cette expérience.

Je remercie tout d'abord la société **Value** de m'avoir accueilli et de m'avoir confié un sujet aussi stimulant, au croisement de l'intelligence artificielle générative et de l'assurance qualité logicielle.

J'adresse mes remerciements les plus sincères à mon encadrant professionnel, **Monsieur Ahmed Rebai**, pour sa disponibilité, ses conseils avisés et la confiance qu'il m'a accordée tout au long de ce stage. Son expertise et son accompagnement ont été déterminants dans l'orientation et la concrétisation de ce travail.

Je remercie également le corps enseignant et l'administration de l'**École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT)** pour la qualité de la formation dispensée, qui m'a permis d'aborder ce projet avec les compétences nécessaires.

Enfin, je tiens à exprimer ma reconnaissance à ma famille et à mes amis pour leur soutien constant et leurs encouragements.

Résumé

Ce rapport présente le travail réalisé durant un stage d'ingénieur de deux mois au sein de la société **Value**. Le projet consiste à concevoir et développer un **système de test autonome** pour les agents conversationnels, fondé sur le framework multi-agents **OpenAI Swarm** et sur des petits modèles de langage (*Small Language Models*, SLM) exécutés localement via **Ollama**.

Le système, baptisé **QA-Swarm**, automatise l'ensemble du cycle de test d'un agent conversationnel financier : génération de scénarios, exécution multi-canal (API REST, Web, Mobile), évaluation de la qualité des réponses (pertinence, exactitude, cohérence, hallucination), production de rapports interactifs et, enfin, **détection de régressions** entre deux versions de l'agent testé. Une console d'administration (*backoffice*) permet de piloter les campagnes de test et de visualiser les tableaux de bord.

La principale contribution de ce travail est une chaîne de test entièrement orchestrée par des agents autonomes (*Generator* → *Executor* → *Evaluator* → *Reporter* → *Fix-Planner*), capable de juger automatiquement la qualité d'un agent conversationnel et de signaler toute dégradation introduite par une nouvelle version.

Mots-clés : test autonome, agents conversationnels, OpenAI Swarm, SLM, Ollama, détection de régression, hallucination, assurance qualité, LLM-as-a-judge.

Abstract

This report describes a two-month engineering internship carried out at **Value**. The project consists in designing and building an **autonomous testing system** for conversational agents, based on the **OpenAI Swarm** multi-agent framework and on **Small Language Models** running locally through **Ollama**. The resulting system, **QA-Swarm**, automates the full testing lifecycle of a financial conversational agent : scenario generation, multi-channel execution (REST API, Web, Mobile), answer-quality evaluation, interactive reporting, and **regression detection** between two versions of the agent under test, all driven by an autonomous agent handoff chain.

Keywords : autonomous testing, conversational agents, OpenAI Swarm, SLM, Ollama, regression detection, hallucination, quality assurance, LLM-as-a-judge.

Table des matières

Remerciements	i
Résumé	ii
Introduction générale	1
1 Contexte général du projet	2
1.1 Introduction	2
1.2 Organisme d'accueil	2
1.3 Cadre du stage	2
1.4 Étude de l'existant et problématique	2
1.4.1 Le système à tester	2
1.4.2 Problématique	2
1.5 Objectifs du projet	3
1.6 Méthodologie de travail	3
1.7 Conclusion	3
2 État de l'art et étude technologique	4
2.1 Introduction	4
2.2 Le test des agents conversationnels	4
2.3 Les petits modèles de langage (SLM) et Ollama	4
2.4 OpenAI Swarm	4
2.5 Outils de test multi-canal	5
2.5.1 API REST	5
2.5.2 Playwright (Web)	5
2.5.3 Appium (Mobile)	5
2.6 Techniques d'évaluation de la qualité textuelle	5
2.7 Conclusion	5
3 Analyse et spécification des besoins	6
3.1 Introduction	6
3.2 Identification des acteurs	6
3.3 Besoins fonctionnels	6
3.4 Besoins non fonctionnels	6
3.5 Le système sous test (SUT)	7
3.6 Conclusion	8
4 Conception du système	9
4.1 Introduction	9
4.2 Architecture globale	9
4.3 Conception des canaux de test	9

4.4	Conception du scoring	10
4.5	Conception de la détection de régression	10
4.6	Conception du planificateur de correctifs	10
4.7	Modèle de données	11
4.8	Conclusion	11
5	Réalisation	12
5.1	Introduction	12
5.2	Environnement de travail	12
5.2.1	Environnement matériel	12
5.2.2	Environnement logiciel	12
5.3	Implémentation des agents et des canaux	13
5.3.1	Connexion de Swarm à Ollama	13
5.3.2	Le canal API	13
5.3.3	L'évaluation hybride	13
5.3.4	La détection de régression	14
5.4	Le corpus de test	14
5.5	La console d'administration (backoffice)	15
5.6	Critères d'évaluation et verdicts	16
5.7	Tests et qualité du code	16
5.8	Conclusion	16
	Conclusion générale et perspectives	17
	Bibliographie et webographie	18

Introduction générale

L'essor des grands modèles de langage (*Large Language Models*, LLM) a fait des agents conversationnels une brique centrale des produits numériques modernes : assistants, chatbots de support, conseillers virtuels. Dans le domaine financier en particulier, un agent conversationnel peut répondre à des questions sur un portefeuille, calculer des indicateurs de performance ou produire des synthèses. Cette puissance s'accompagne toutefois d'un risque majeur : l'**imprévisibilité**. Contrairement à un logiciel déterministe, un agent fondé sur un modèle de langage peut fournir des réponses différentes à la même question, inventer des faits (*hallucinations*), ou voir sa qualité se dégrader silencieusement lorsqu'on modifie son *prompt*, son modèle sous-jacent ou son code de routage.

Les méthodes de test traditionnelles, fondées sur des assertions exactes (*entrée* → *sortie attendue*), sont mal adaptées à ce type de système : il n'existe pas une seule « bonne » réponse, mais un espace de réponses acceptables. Tester un agent conversationnel impose donc d'**évaluer la qualité sémantique** d'une réponse plutôt que sa stricte égalité à une référence, et de le faire à grande échelle, de manière reproductible, et sur tous les canaux par lesquels l'utilisateur accède au service.

C'est dans ce contexte que s'inscrit le présent stage. L'objectif est de concevoir un **système de test autonome** capable de tester de bout en bout un agent conversationnel financier, sur ses trois surfaces d'accès (API REST, application Web, application Mobile), d'évaluer automatiquement la qualité de ses réponses, et surtout de **détecter les régressions** entre deux versions successives de l'agent.

Le système développé, **QA-Swarm**, repose sur le framework multi-agents **OpenAI Swarm** et sur des petits modèles de langage exécutés localement via **Ollama**, ce qui garantit confidentialité et coût nul d'inférence. L'agent conversationnel testé, nommé *La-Ruche*, est une plateforme de gestion de patrimoine construite comme un maillage (*mesh*) d'agents spécialisés ; il joue le rôle de **système sous test** (*System Under Test*, SUT).

Ce rapport est organisé en cinq chapitres. Le **premier** présente le contexte général du projet : l'entreprise d'accueil, la problématique et la méthodologie de travail. Le **deuxième** dresse une étude de l'état de l'art du test des agents conversationnels et des technologies retenues. Le **troisième** formalise l'analyse et la spécification des besoins. Le **quatrième** détaille la conception de l'architecture du système. Le **cinquième** décrit la réalisation et les résultats obtenus. Une conclusion générale et des perspectives clôturent le document.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce premier chapitre situe le projet dans son cadre. Il présente l'organisme d'accueil, expose la problématique à l'origine du sujet, définit les objectifs du stage et décrit la méthodologie de travail adoptée.

1.2 Organisme d'accueil

Ce stage s'est déroulé au sein de la société **Value**, une entreprise active dans le domaine de l'ingénierie logicielle et de l'intelligence artificielle. Value accompagne ses clients dans la conception de solutions logicielles innovantes, avec un intérêt marqué pour les technologies d'IA générative et leur industrialisation. Dans ce cadre, l'entreprise s'intéresse aux enjeux de fiabilité et de qualité des systèmes fondés sur des modèles de langage, qui constituent le cœur du présent projet.

1.3 Cadre du stage

Le stage, d'une durée de **deux mois** (du 12 juin 2026 au 12 août 2026), s'inscrit dans le cursus d'ingénieur de l'ESPRIT. Il correspond à un stage d'ingénieur visant à mettre en application les compétences acquises en développement logiciel, en intelligence artificielle et en génie logiciel, dans un contexte professionnel réel.

1.4 Étude de l'existant et problématique

1.4.1 Le système à tester

L'objet du test est un agent conversationnel financier, *LaRuche*, conçu comme un **maillage d'agents** (*agent mesh*). Une requête utilisateur est reçue par un **orchestrateur** qui la route, selon son intention, vers un ou plusieurs agents spécialisés (financier, marché, documents, action, qualité), puis agrège les réponses et les renvoie en flux continu (*streaming*). Cet agent est accessible par trois canaux distincts : une **API REST**, une **application Web** et une **application Mobile**.

1.4.2 Problématique

La nature non déterministe de cet agent soulève plusieurs difficultés que les approches de test classiques ne savent pas traiter :

- **Absence de sortie unique de référence** : à une même question peuvent correspondre plusieurs réponses correctes formulées différemment. Une comparaison exacte n'a donc pas de sens.
- **Risque d'hallucination** : l'agent peut inventer des chiffres ou des faits non fondés, ce qui est particulièrement grave dans un contexte financier.
- **Régressions silencieuses** : une modification du *prompt*, du modèle ou du code de routage peut dégrader la qualité sans provoquer d'erreur visible.
- **Multiplicité des canaux** : le même agent doit se comporter de manière cohérente sur l'API, le Web et le Mobile.
- **Robustesse** : l'agent doit résister aux entrées limites (vides, très longues) et aux attaques (injection de *prompt*, XSS, questions hors domaine).

La question centrale du stage est donc la suivante : *comment tester de manière automatique, reproductible et multi-canal un agent conversationnel, en évaluant la qualité sémantique de ses réponses et en détectant toute régression entre deux versions ?*

1.5 Objectifs du projet

Pour répondre à cette problématique, le projet vise les objectifs suivants :

1. Concevoir une **chaîne de test autonome** orchestrée par des agents (génération, exécution, évaluation, rapport, planification de correctifs).
2. Couvrir les **trois canaux** d'accès : API REST, Web et Mobile.
3. Évaluer automatiquement la qualité des réponses selon des critères sémantiques (pertinence, exactitude, cohérence) et détecter les hallucinations.
4. Produire des **rapports** lisibles et des tableaux de bord interactifs.
5. Implémenter la **détection de régression** entre deux versions de l'agent — critère de succès central du projet.
6. Offrir une **console d'administration** pour piloter les campagnes.

1.6 Méthodologie de travail

Le projet a été conduit selon une démarche **itérative et incrémentale**, inspirée des méthodes agiles. Le travail a été découpé en phases successives, chacune livrant un incrément fonctionnel et vérifiable : mise en place de l'infrastructure et du lien Swarm/Ollama, puis canal API, puis canaux Web et Mobile, puis métriques et tableaux de bord, puis détection de régression, et enfin console d'administration. Le code est versionné avec **Git** et chaque composant est accompagné de tests automatisés.

1.7 Conclusion

Ce chapitre a posé le cadre du stage et formulé la problématique : tester de façon autonome et multi-canal un agent conversationnel non déterministe. Le chapitre suivant étudie l'état de l'art et justifie les choix technologiques.

Chapitre 2

État de l’art et étude technologique

2.1 Introduction

Ce chapitre présente les concepts et technologies sur lesquels repose le projet : la spécificité du test des agents conversationnels, les petits modèles de langage, le framework OpenAI Swarm, les outils de test multi-canal et les techniques d’évaluation de la qualité.

2.2 Le test des agents conversationnels

Tester un système fondé sur un LLM diffère fondamentalement du test logiciel classique. On distingue plusieurs niveaux :

- **Test fonctionnel** : vérifier que l’agent répond à l’intention de l’utilisateur.
- **Test de qualité sémantique** : mesurer la pertinence, l’exactitude et la cohérence de la réponse plutôt que son égalité exacte à une référence.
- **Test de robustesse** : éprouver l’agent sur des entrées limites et des prompts adverses.
- **Test de non-régression** : garantir qu’une nouvelle version ne dégrade pas le comportement existant.

La difficulté majeure réside dans l’évaluation automatique : faute de sortie de référence unique, on recourt de plus en plus au paradigme **LLM-as-a-judge**, où un modèle de langage joue le rôle de juge et note la réponse selon une grille de critères.

2.3 Les petits modèles de langage (SLM) et Ollama

Les **Small Language Models** (SLM) sont des modèles de langage de taille réduite (quelques milliards de paramètres) qui peuvent s’exécuter localement sur du matériel grand public. Ils offrent plusieurs avantages décisifs pour un système de test : **confidentialité** (aucune donnée ne quitte la machine), **coût nul** d’inférence et **reproductibilité**.

Ollama est un environnement d’exécution qui permet de télécharger et de servir des SLM localement en exposant une API compatible avec celle d’OpenAI. Le projet utilise principalement le modèle **qwen2.5:3b**, retenu pour sa fiabilité dans l’appel d’outils (*tool-calling*), indispensable aux passages de relais (*handoffs*) du framework Swarm, et pour sa compacité (il tient intégralement dans 4 Go de mémoire vidéo).

2.4 OpenAI Swarm

OpenAI Swarm est un framework expérimental d’orchestration multi-agents. Il repose sur deux primitives simples :

- les **Agents**, définis par un nom, un modèle, des instructions (*system prompt*) et un ensemble de fonctions ;
- les **handoffs**, mécanisme par lequel un agent passe le relais à un autre en retournant simplement une référence vers cet agent.

Cette légèreté en fait un candidat idéal pour modéliser une chaîne de test où chaque étape (génération, exécution, évaluation, rapport) est prise en charge par un agent spécialisé, selon le principe « **un agent = un SLM** ». Swarm communiquant via une API compatible OpenAI, il se branche directement sur Ollama.

2.5 Outils de test multi-canal

2.5.1 API REST

Le test de l'API se fait par requêtes HTTP directes. Le client **httpx** (Python) permet d'envoyer des requêtes et de consommer un flux **Server-Sent Events** (SSE), protocole utilisé par l'agent pour renvoyer ses réponses jeton par jeton.

2.5.2 Playwright (Web)

Playwright est un framework moderne d'automatisation de navigateur. Plus rapide et plus fiable que l'historique Selenium grâce à son attente automatique des éléments, il pilote un navigateur *Chromium* pour simuler un utilisateur réel sur l'application Web.

2.5.3 Appium (Mobile)

Appium est le standard de l'automatisation des applications mobiles. Via le pilote *UiAutomator2*, il contrôle l'application sur un émulateur Android et y reproduit les interactions de l'utilisateur.

2.6 Techniques d'évaluation de la qualité textuelle

Outre le jugement par LLM, le projet emploie des métriques classiques de comparaison de texte :

- **BLEU** (via **sacrebleu**) et **ROUGE-L** (via **rouge-score**), qui mesurent le recouvrement lexical entre la réponse obtenue et la réponse attendue ;
- la **similarité de séquence** (**difflib**), utilisée pour mesurer la **divergence** entre les réponses des canaux Web et Mobile.

2.7 Conclusion

L'étude technologique a montré la pertinence d'une approche fondée sur des agents autonomes (Swarm) et des SLM locaux (Ollama), complétée par des outils de test éprouvés (**httpx**, **Playwright**, **Appium**) et des techniques d'évaluation sémantique. Le chapitre suivant formalise les besoins du système.

Chapitre 3

Analyse et spécification des besoins

3.1 Introduction

Ce chapitre identifie les acteurs du système, recense les besoins fonctionnels et non fonctionnels, et précise le périmètre du système sous test.

3.2 Identification des acteurs

- **L'ingénieur QA / développeur** : déclenche les campagnes de test, consulte les rapports et les plans de correctifs, compare deux versions de l'agent.
- **Le système d'intégration continue (CI)** : exécute automatiquement un test de fumée (*smoke test*) à chaque modification, afin de bloquer toute régression bloquante avant fusion.
- **L'agent sous test (*LaRuche*)** : acteur secondaire, sollicité par le système de test sur ses trois canaux.

3.3 Besoins fonctionnels

Le système doit permettre de :

1. **Générer** un corpus de scénarios de test (nominaux, limites, adverses), versionné en JSON, et en synthétiser de nouveaux à partir d'une spécification.
2. **Exécuter** chaque scénario sur les canaux ciblés (API, Web, Mobile).
3. **Évaluer** la réponse selon la pertinence, l'exactitude, la cohérence et la présence d'hallucinations, puis rendre un verdict PASS/FAIL.
4. **Mesurer** la latence, le temps de première réponse, la disponibilité et la divergence entre canaux.
5. **Produire** un rapport Markdown et un tableau de bord HTML interactif.
6. **Détecter les régressions** entre deux exécutions (passages PASS→FAIL, chutes de score, hausses de latence).
7. **Proposer** des plans de correctifs non destructifs pour les échecs.
8. **Piloter** l'ensemble depuis une console d'administration Web.

3.4 Besoins non fonctionnels

- **Confidentialité** : l'inférence doit être 100 % locale (aucune donnée envoyée à un service tiers).

- **Reproductibilité** : les scénarios sont versionnés et les exécutions horodatées et persistées.
- **Modularité** : chaque agent et chaque canal doit être indépendant et remplaçable.
- **Sécurité** : le planificateur de correctifs ne doit jamais modifier le code (principe de non-destruction).
- **Robustesse** : un échec sur un canal ne doit pas interrompre la campagne.
- **Configurabilité** : URLs, modèles et seuils sont paramétrables par variables d'environnement.

3.5 Le système sous test (SUT)

L'agent *LaRuche* est composé de sept services indépendants, présentés au tableau 3.1. C'est sur cet ensemble que s'exerce le système de test, en priorité via le point d'entrée POST `/api/chat` de l'orchestrateur.

TABLE 3.1 – Services composant l'agent sous test (*LaRuche*).

Service	Port	Rôle
Orchestrateur	8000	Routage des requêtes, agrégation, flux SSE
Agent financier	8001	Indicateurs de portefeuille (AUM, TWR, IRR, Sharpe)
Agent marché	8002	Données de marché et cotations
Agent documents	8003	Recherche documentaire (RAG)
Agent action	8004	Actions (envoi de courriel, messages)
Agent QA	8005	Génération et exécution de tests internes
Service vocal	8006	Transcription et synthèse vocale

Les figures 3.1 et 3.2 illustrent l'interface utilisateur du SUT : le tableau de bord de portefeuille et l'assistant conversationnel.



FIGURE 3.1 – Tableau de bord du SUT *LaRuche* — vue portefeuille avec indicateurs financiers.

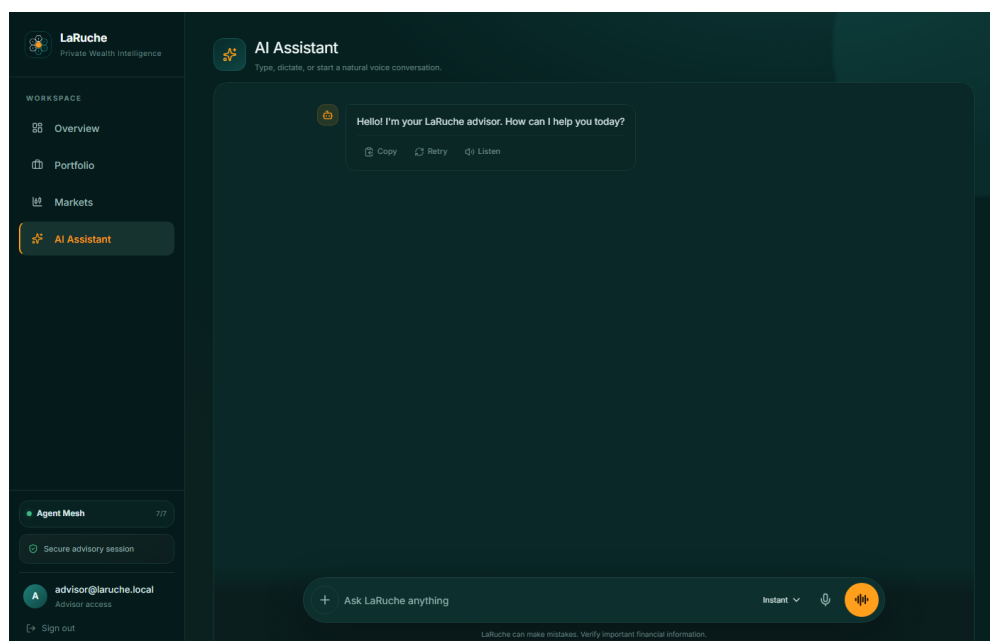


FIGURE 3.2 – Interface de l'assistant conversationnel du SUT (Agent Mesh 7/7).

3.6 Conclusion

Les besoins étant clarifiés et le périmètre du SUT défini, le chapitre suivant en présente la conception détaillée.

Chapitre 4

Conception du système

4.1 Introduction

Ce chapitre décrit l’architecture globale de QA-Swarm, le rôle de chaque agent, la conception des canaux, le mécanisme de scoring, la détection de régression et le modèle de données.

4.2 Architecture globale

QA-Swarm est organisé comme une **chaîne d’agents autonomes** reliés par des passages de relais. Le flux est le suivant :

Generator → Executor → Evaluator → Reporter → FixPlanner

Chaque agent est un agent Swarm doté de son propre SLM (servi par Ollama) et d’un rôle unique. Le tableau 4.1 récapitule ces rôles.

TABLE 4.1 – Les agents de la chaîne QA-Swarm.

Agent	Rôle	Modèle
Generator	Charge le corpus JSON ou synthétise des scénarios	qwen2.5 :3b
Executor	Dispatche chaque cas vers API, Web ou Mobile	qwen2.5 :3b
Evaluator	Note pertinence, exactitude, cohérence, hallucination	qwen2.5 :3b
Reporter	Produit les rapports Markdown et HTML (Plotly)	qwen2.5 :3b
FixPlanner	Classe les échecs et écrit des plans de correctifs sûrs	heuristiques / qwen2.5 :3b

4.3 Conception des canaux de test

Trois canaux partagent une même interface (`ChannelResult`) mais diffèrent dans leur exécution :

- **Canal API** : envoi direct d’une requête HTTP sur `/api/chat`, consommation du flux SSE, mesure de la latence et du temps de première réponse (*TTFT*). Pur `httpx`, sans interface ni LLM.

- **Canal Web** : selon la philosophie « *code-based* », le SLM de l'*Executor* **génère un script Playwright complet** pour le scénario. Ce script est validé par `ast.parse` puis exécuté dans un **bac à sable** (sous-processus avec délai limite). Un script de repli éprouvé est utilisé si la génération échoue.
- **Canal Mobile** : même principe *code-based* avec un script Appium ciblant l'émulateur Android.

4.4 Conception du scoring

L'évaluation combine un **juge LLM** et des **règles déterministes**. Les règles d'échec **bloquantes** sont appliquées en premier, sans appel au modèle :

- dépassement du délai (*timeout*, 30 s) ;
- plantage ou erreur serveur (5xx) ;
- réponse vide ;
- hallucination détectée.

En l'absence de ces conditions, le juge LLM note la réponse sur trois axes (pertinence, exactitude, cohérence) de 1 à 5 et signale toute hallucination. Le verdict final est **calculé de façon déterministe** : une hallucination entraîne un échec ; sinon, la réponse passe si la moyenne des trois scores atteint le seuil de 3,0. Ce choix — laisser le LLM noter mais décider la règle en dur — est plus fiable que de confier l'application de la règle à un modèle de 3 milliards de paramètres.

4.5 Conception de la détection de régression

La détection de régression est le **critère de succès** du projet. Elle compare deux exécutions complètes (*RunReport*), une *baseline* et une *candidate*, appariées par intention et par canal. Trois types de régression sont signalés :

- **Bascule PASS→FAIL** : un scénario qui passait échoue désormais ;
- **Chute de score** : baisse de la moyenne d'au moins 0,5 point ;
- **Pic de latence** : hausse de la latence d'au moins 50 %.

4.6 Conception du planificateur de correctifs

Le **FixPlanner** est volontairement **non destructif** : il n'écrit jamais de code. Pour chaque échec, il produit un **triage** (catégorie, sévérité, confiance, propriétaire probable, preuves) et un **plan de correctif** (hypothèse de cause racine, fichiers probables, étapes, commandes de vérification, notes de retour arrière). Tout plan porte les drapeaux `auto_apply=false` et `human_approval_required=true`.

4.7 Modèle de données

Les structures de données, validées par **Pydantic**, garantissent la cohérence de toute la chaîne. Les principales entités sont résumées au tableau 4.2.

TABLE 4.2 – Principales structures de données.

Entité	Description
TestCase	Un scénario : id, intention, type, entrée, attendu, canaux
ChannelResult	Résultat brut sur un canal : réponse, statut, latence, TTFT, drapeaux
Score	Notes pertinence/exactitude/cohérence, hallucination, verdict
ScenarioResult	Un cas + ses résultats et scores par canal + divergence
RunReport	Une exécution complète (unité comparée pour la régression)
FailureAnalysis	Triage + plan de correctif d'un échec

4.8 Conclusion

La conception ayant fixé l'architecture, les agents, les canaux et les mécanismes d'évaluation, le chapitre suivant en présente la réalisation concrète.

Chapitre 5

Réalisation

5.1 Introduction

Ce chapitre présente l’environnement technique, des extraits significatifs de l’implémentation, le corpus de test, la console d’administration et les résultats obtenus.

5.2 Environnement de travail

5.2.1 Environnement matériel

Le développement a été mené sur une station de travail équipée d’un processeur multi-cœurs, de 16 Go de mémoire vive et d’une carte graphique disposant de 4 Go de mémoire vidéo, suffisante pour exécuter le modèle `qwen2.5:3b` via Ollama.

5.2.2 Environnement logiciel

Le tableau 5.1 récapitule les principaux outils et technologies utilisés.

TABLE 5.1 – Environnement logiciel du projet.

Catégorie	Outils
Langage	Python 3.12
Orchestration d’agents	OpenAI Swarm
Exécution des modèles	Ollama (<code>qwen2.5:3b</code>)
Tests de canaux	httpx (API), Playwright (Web), Appium (Mobile)
Validation de données	Pydantic
Métriques NLP	sacrebleu, rouge-score, difflib
Rapports	Markdown, Plotly (HTML)
Backoffice	FastAPI, Jinja2
Tests unitaires	pytest, respx
Gestion de paquets	uv
Versionnement	Git

5.3 Implémentation des agents et des canaux

5.3.1 Connexion de Swarm à Ollama

Chaque agent Swarm pointe vers le point d'accès compatible OpenAI exposé par Ollama, ce qui permet d'exécuter les modèles localement sans clé d'API distante.

```

1 from swarm import Agent
2
3 def build_evaluator_agent() -> Agent:
4     return Agent(
5         name="Evaluator",
6         model=settings.model_evaluator, # qwen2.5:3b via Ollama
7         instructions=EVALUATOR_INSTRUCTIONS,
8     )

```

Listing 5.1 – Définition d'un agent Swarm (Evaluator).

5.3.2 Le canal API

Le canal API envoie la requête et consomme le flux SSE jeton par jeton, en mesurant la latence et le temps de première réponse.

```

1 with httpx.Client(timeout=settings.timeout_seconds) as client, \
2     client.stream("POST", settings.sut_chat_url, json=payload,
3                 headers=headers) as r:
4     status_code = r.status_code
5     for line in r.iter_lines():
6         if not line or not line.startswith("data: "):
7             continue
8         data = line[6:]
9         if data.strip() == "[DONE]":
10            break
11        obj = json.loads(data)
12        token = obj.get("token") or obj.get("reasoning") or ""
13        if token:
14            if ttft_ms is None:
15                ttft_ms = round((time.perf_counter() - start) * 1000, 1)
16            tokens.append(token)

```

Listing 5.2 – Cœur du canal API : consommation du flux SSE.

5.3.3 L'évaluation hybride

Le scoring applique d'abord les règles bloquantes, puis le jugement LLM, et calcule enfin le verdict de manière déterministe.

```

1 # 1. Règles bloquantes déterministes (aucun appel LLM)
2 if result.timed_out or result.crashed:
3     return Score(verdict="FAIL", reason="timeout/crash", format_ok=False)
4 if not result.actual.strip():
5     return Score(verdict="FAIL", reason="empty reply", format_ok=False)
6
7 # 2. Jugement LLM (LLM-as-a-judge)
8 judged = _judge(test_case, result)
9 score = Score(pertinence=..., exactitude=..., coherence=...,
10              hallucination=bool(judged.get("hallucination", False)))
11
12 # 3. Verdict déterministe
13 if score.hallucination:

```

```

14     score.verdict = "FAIL"
15 elif score.mean >= settings.min_score_pass:    # seuil = 3.0
16     score.verdict = "PASS"
17 else:
18     score.verdict = "FAIL"

```

Listing 5.3 – Logique de verdict de l'Evaluator.

5.3.4 La détection de régression

La comparaison de deux exécutions repère les bascules, chutes de score et pics de latence.

```

1 score_delta = round(score.mean - base_score.mean, 2)
2 latency_delta = round(cand_latency - base_latency, 1)
3 is_flip = base_score.verdict == "PASS" and score.verdict == "FAIL"
4 is_score_drop = score_delta <= -score_drop_threshold    # -0.5
5 is_latency_spike = (base_latency > 0 and latency_delta > 0
6     and (latency_delta / base_latency) * 100 >= latency_spike_pct)    # +50%

```

Listing 5.4 – Détection des trois types de régression.

5.4 Le corpus de test

Le corpus, versionné en JSON, compte **52 scénarios** couvrant les intentions réelles de l'agent. Sa répartition par type et par canal est donnée aux tableaux 5.2 et 5.3.

TABLE 5.2 – Répartition du corpus par type de scénario.

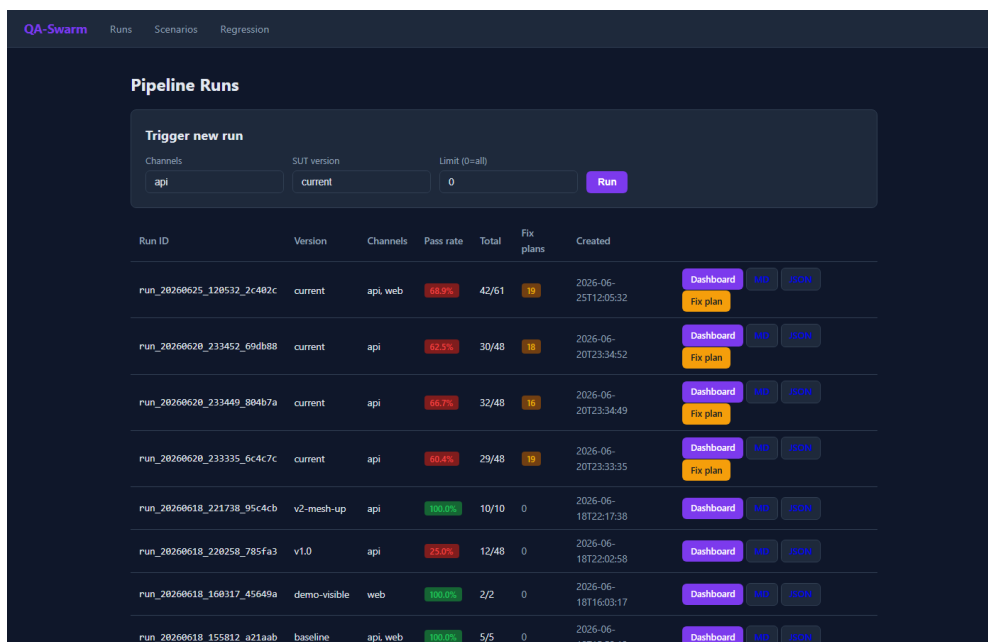
Type	Nombre	Description
Nominal	30	Questions financières normales (AUM, TWR, IRR, allocations...)
Limite	10	Entrées vides, très longues, cas aux bords
Adverse	12	Injection de prompt, XSS, questions hors domaine
Total	52	

TABLE 5.3 – Couverture du corpus par canal.

Canal	Scénarios	Nature du test
API	48	Test rapide, latence et flux SSE
Web	13	Parcours utilisateur via Playwright
Mobile	5	Parcours via Appium sur émulateur Android

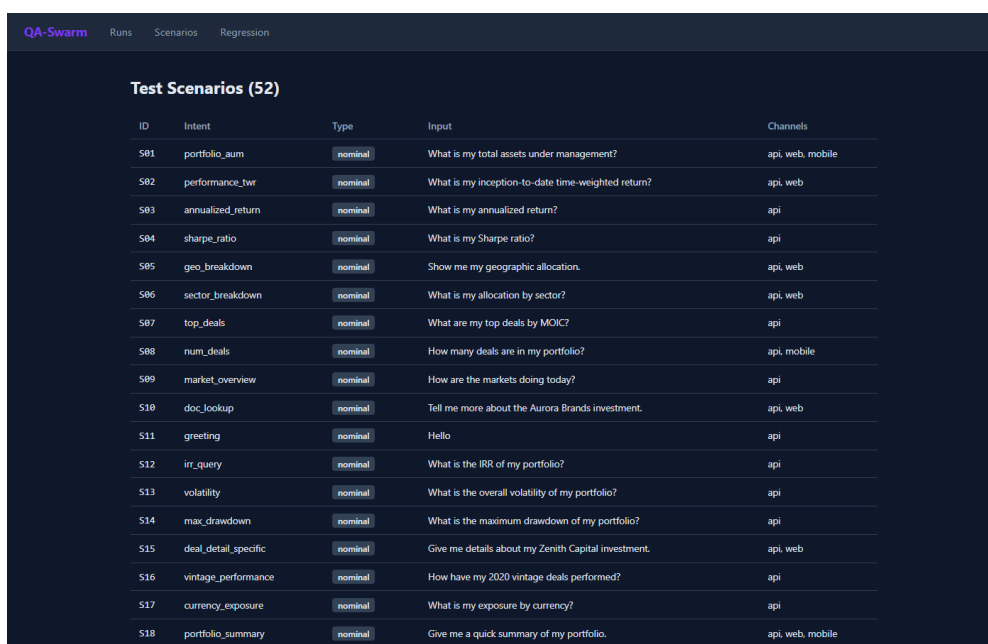
5.5 La console d'administration (backoffice)

Une application **FastAPI** (port 8090) offre une interface Web permettant de déclencher des campagnes, parcourir les scénarios, visualiser les tableaux de bord interactifs (Plotly), comparer deux exécutions et consulter les plans de correctifs. Les figures 5.1 et 5.2 présentent les deux vues principales de cette console.



Run ID	Version	Channels	Pass rate	Total	Fix plans	Created	Dashboard	MD	JSON	Fix plan
run_20260625_128532_2c482c	current	api, web	68.9%	42/61	19	2026-06-25T12:05:32	Dashboard	MD	JSON	Fix plan
run_20260620_233452_69db88	current	api	62.5%	30/48	18	2026-06-20T23:34:52	Dashboard	MD	JSON	Fix plan
run_20260620_233449_804b7a	current	api	66.7%	32/48	16	2026-06-20T23:34:49	Dashboard	MD	JSON	Fix plan
run_20260620_233335_6c4c7c	current	api	60.4%	29/48	19	2026-06-20T23:33:35	Dashboard	MD	JSON	Fix plan
run_20260618_221738_95c4cb	v2-mesh-up	api	100.0%	10/10	0	2026-06-18T22:17:38	Dashboard	MD	JSON	Fix plan
run_20260618_220258_785fa3	v1.0	api	25.0%	12/48	0	2026-06-18T22:02:58	Dashboard	MD	JSON	Fix plan
run_20260618_160317_45649a	demo-visible	web	100.0%	2/2	0	2026-06-18T16:03:17	Dashboard	MD	JSON	Fix plan
run_20260618_155812_a21aab	baseline	api, web	100.0%	5/5	0	2026-06-18T15:58:12	Dashboard	MD	JSON	Fix plan

FIGURE 5.1 – Console QA-Swarm — liste des exécutions avec taux de réussite et liens vers les tableaux de bord, rapports Markdown/JSON et plans de correctifs.



ID	Intent	Type	Input	Channels
S01	portfolio_aum	nominal	What is my total assets under management?	api, web, mobile
S02	performance_twr	nominal	What is my inception-to-date time-weighted return?	api, web
S03	annualized_return	nominal	What is my annualized return?	api
S04	sharpe_ratio	nominal	What is my Sharpe ratio?	api
S05	geo_breakdown	nominal	Show me my geographic allocation.	api, web
S06	sector_breakdown	nominal	What is my allocation by sector?	api, web
S07	top_deals	nominal	What are my top deals by MOIC?	api
S08	num_deals	nominal	How many deals are in my portfolio?	api, mobile
S09	market_overview	nominal	How are the markets doing today?	api
S10	doc_lookup	nominal	Tell me more about the Aurora Brands investment.	api, web
S11	greeting	nominal	Hello	api
S12	irr_query	nominal	What is the IRR of my portfolio?	api
S13	volatility	nominal	What is the overall volatility of my portfolio?	api
S14	max_drawdown	nominal	What is the maximum drawdown of my portfolio?	api
S15	deal_detail_specific	nominal	Give me details about my Zenith Capital investment.	api, web
S16	vintage_performance	nominal	How have my 2020 vintage deals performed?	api
S17	currency_exposure	nominal	What is my exposure by currency?	api
S18	portfolio_summary	nominal	Give me a quick summary of my portfolio.	api, web, mobile

FIGURE 5.2 – Corpus de 52 scénarios de test classés par type (nominal, limite, adversarial) et par canaux ciblés (API, Web, Mobile).

5.6 Critères d'évaluation et verdicts

Le tableau 5.4 récapitule les seuils de décision configurés.

TABLE 5.4 – Seuils de décision du système.

Critère	Seuil
Délai maximal d'une réponse (timeout)	30 secondes
Score moyen minimal pour réussir	3,0 / 5
Divergence Web↔Mobile tolérée	20 %
Chute de score signalée comme régression	$\geq 0,5$ point
Pic de latence signalé comme régression	≥ 50 %

5.7 Tests et qualité du code

Le système de test est lui-même testé : la suite `pytest` compte **19 fonctions de test** couvrant le canal API (avec un SUT simulé via `respx`), l'extraction JSON, la chaîne de pipeline, la détection de régression et le planificateur de correctifs. Un script de **test de fumée** (`ci_smoke.py`) sert de garde-fou en intégration continue : il bloque toute fusion introduisant un échec bloquant.

5.8 Conclusion

Ce chapitre a montré la mise en œuvre concrète du système : agents, canaux, scoring, détection de régression, corpus, console et tests. Le système répond à l'ensemble des besoins fixés.

Conclusion générale et perspectives

Ce stage avait pour objectif de concevoir et développer un système de test autonome pour agents conversationnels. Le résultat, **QA-Swarm**, est une chaîne de test entièrement orchestrée par des agents autonomes — *Generator*, *Executor*, *Evaluator*, *Reporter* et *FixPlanner* — bâtie sur le framework **OpenAI Swarm** et des petits modèles de langage exécutés localement via **Ollama**.

Le système atteint les objectifs fixés : il teste l'agent sur ses **trois canaux** (API, Web, Mobile), évalue automatiquement la **qualité sémantique** des réponses (pertinence, exactitude, cohérence, hallucination), produit des **rapports** et des tableaux de bord interactifs, et — point central — **détecte les régressions** entre deux versions de l'agent. Une console d'administration permet de piloter l'ensemble, et le *FixPlanner* ajoute une dimension de diagnostic tout en respectant un principe strict de non-destruction.

Au-delà du livrable, ce stage a été l'occasion de monter en compétence sur des sujets de pointe : l'orchestration multi-agents, l'exploitation locale de modèles de langage, le paradigme *LLM-as-a-judge* et les enjeux spécifiques de l'assurance qualité des systèmes d'IA générative. Il m'a également permis de consolider mes pratiques de génie logiciel : conception modulaire, tests automatisés, intégration continue et documentation.

Plusieurs **perspectives** d'amélioration se dégagent :

- enrichir le corpus au-delà de 52 scénarios et automatiser davantage sa génération agentique ;
- renforcer le canal Mobile et industrialiser l'exécution sur une ferme d'émulateurs ;
- ajouter des métriques d'évaluation supplémentaires (cohérence multi-tours, détection fine de biais) ;
- intégrer la chaîne complète dans un *pipeline* CI/CD de bout en bout, avec historisation des tendances de qualité dans le temps ;
- faire évoluer le *FixPlanner* vers une proposition de correctifs soumise à validation humaine.

En définitive, ce projet illustre comment des agents autonomes peuvent être mis au service de la fiabilité d'autres systèmes d'IA, ouvrant la voie à une assurance qualité elle-même intelligente et autonome.

Bibliographie et webographie

1. OpenAI, *Swarm : an educational framework for lightweight multi-agent orchestration*, <https://github.com/openai/swarm>.
2. Ollama, *Get up and running with large language models locally*, <https://ollama.com>.
3. Qwen Team, *Qwen2.5 Technical Report*, Alibaba Cloud, 2024.
4. Microsoft, *Playwright — Fast and reliable end-to-end testing for modern web apps*, <https://playwright.dev>.
5. Appium, *Cross-platform automation framework for all kinds of apps*, <https://appium.io>.
6. Zheng, L. *et al.*, *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*, NeurIPS, 2023.
7. Pydantic, *Data validation using Python type hints*, <https://docs.pydantic.dev>.
8. Sebastián Ramírez, *FastAPI — modern, fast web framework for building APIs with Python*, <https://fastapi.tiangolo.com>.